Lab 7: Design and FPGA Implementation of the Register File for a Single-Cycle RISC-V Processor

Objective

To design and implement the 32×32 register file (RV32I convention) using Verilog HDL, verify its functionality through a simulation testbench, synthesize the design on an FPGA, and demonstrate its operation by building an FSM that exercises both the ALU and the register file together. The register file should follow RISC-V conventions (register x0 hardwired to 0, two asynchronous read ports, one synchronous write port) and the FSM should perform a sequence of reads/writes and ALU operations to validate correct integration.

Task 1: Register File Block Diagram Design

Objective

To create a block diagram of the register file showing inputs, outputs, internal features (x0 behavior), and how it connects to the ALU.

Procedure

(a) Design the full 32×32 register file block diagram showing:

- Inputs: `clk`, `rst`, `WriteEnable`, `rs1[4:0]`, `rs2[4:0]`, `rd[4:0]`, `WriteData[31:0]`
- Outputs: `ReadData1[31:0]`, `readData2[31:0]`
- Internal x0 handling: address 0 always returns 0 and writes to address 0 are ignored.

(b) Draw clear diagrams showing data flow:

- Show how `writeData` is routed to the selected register on a rising clock edge when `WriteEnable` is asserted.
- Show how `rs1` and `rs2` select registers for combinational outputs `readData1` and `reaData2`.
- Show connection points to the ALU: `readData1` and `readData2` feed the ALU's `A` and `B` inputs (or one of them).

Deliverables

- Block diagram of the 32×32 register file and its integration points with the ALU.

Task 2: Verilog Description and Testbench Development

Objective

Implement the register file in Verilog based on the block diagram and verify its functionality using simulation. Then develop an integrated testbench that uses an FSM to test the ALU and register file together.

Procedure

(a) Implement the Register File module in Verilog:

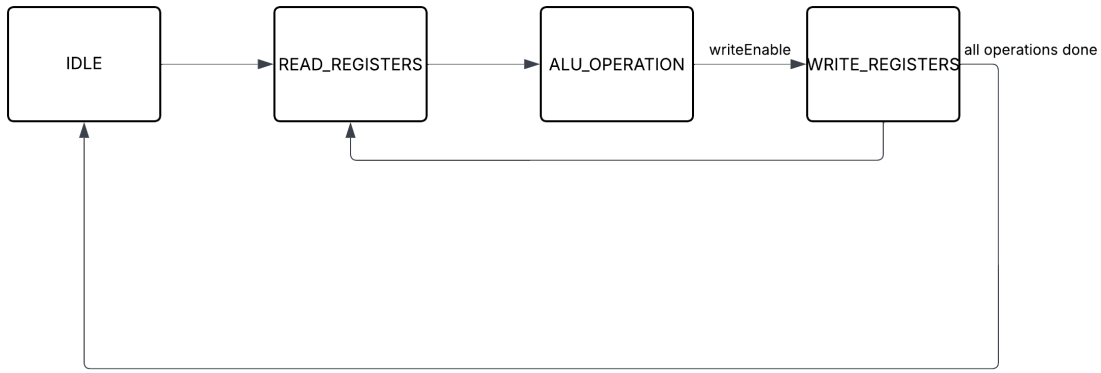
- Enforce RISC-V semantics: register x0 (address 0) always reads as 32'b0 and writes to x0 are ignored.
- Use an array of registers (`reg [31:0] regs [31:0]` or similar)

(b) Develop a testbench for the register file only (`RegisterFile_tb.v`):

- Test cases:
  i. Write a value to a register (e.g., x5 = 0xDEADBEEF) with `WriteEnable=1` and check on next clock that `readData1` or `readData2` returns the same when addressed.
  ii. Attempt to write to x0 and verify it remains zero.
  iii. Simultaneous two read ports: verify both reads return the correct contents concurrently.
  iv. Overwrite a register and verify old value is replaced.
  v. Reset behavior: if a synchronous/asynchronous reset is implemented, check registers clear appropriately.
- Generate waveforms.

(c) Implement an integrated testbench that exercises the Register File and ALU together with an FSM (`RF_ALU_FSM_tb.v`):

- Instantiate the previously implemented ALU module (from the ALU lab) and the Register File.
- Build an FSM that cycles through a deterministic sequence of states to:
  i. Write known constants into several registers (e.g., x1 = 0x10101010, x2 = 0x01010101, x3 = 0x00000005).
  ii. Read x1 and x2, feed them to the ALU, perform ADD, SUB, AND, OR, XOR, SLL, SRL operations, and write ALU results into destination registers (e.g., x4..x10).
  iii. Perform a BEQ-style check using ALU Zero output and conditionally write a flag register.
  iv. Test register read-after-write timing (write a register and then read it in the following cycle).
- The FSM should include clearly named states (e.g., `IDLE`, `WRITE_REGS`).



- Produce waveforms capturing the register file contents, ALU inputs/outputs, and FSM state transitions.

(d) Run simulation to ensure correctness of all behaviors:

- Register read/write correctness and x0 behavior.
- ALU results when fed from register file outputs.
- FSM transitions and conditional actions based on ALU Zero flag.

Deliverables

- Verilog source file for Register File (`RegisterFile.v`)
- Testbench file for Register File (`RegisterFile_tb.v`)
- Integrated FSM + ALU + Register File testbench (`RF_ALU_FSM_tb.v`)
- Simulation waveforms showing correct operation and assertions printed for passed tests

Task 3: FPGA Synthesis and RTL Analysis

Objective

Synthesize the Register File and the FSM+ALU integration on an FPGA, analyze the RTL structure, and verify functionality using switches, LEDs, and (optionally) a small seven-segment display for numeric output.

Procedure

(a) Integrate Register File and ALU into a top-level design (`top_rf_alu.v` or similar):

- Instantiate the Register File and the ALU.
- Add an FSM module that repeats the verification sequence used in simulation.
- Map switches to force `WriteEnable`, `rd`

(b) Map inputs and outputs to FPGA I/O:

- Map a small set of switches to choose which test or which ALU operation to run, and to set control values including `writeEnable`.
- Map LEDs to show FSM state.

(c) Use two fixed 32-bit constants for quick verification (example):

- Constant A = `32'h10101010`
- Constant B = `32'h01010101`
- Use FSM to write these into registers (e.g., x1 and x2) at startup.

(d) Synthesize the design and analyze the RTL schematic:

- Inspect the register array implementation, write-enable logic, and decoder.

(e) Program the FPGA and verify on-board:

- Press the configured switches to choose an ALU operation and observe LED patterns showing the result (lower bits) and the FSM state.
- Run the FSM demo sequence and demonstrate to RA.

Deliverables

- RTL schematic screenshots (register file array, decoder, FSM, ALU connections)
- Demonstration of FPGA operation to RA for approval
- Top-level Verilog file (`top_rf_alu.v`).

Note: The switch interface, button debouncing module, and LED interface must be re used from Lab 5.

Assessment Rubric
Lab 7: Design and FPGA Implementation of the Register File for a Single-Cycle RISC-V Processor

Name:	Student ID:	Section:

Points Distribution

	Task No.	LR 2 (Code)	LR 5 (Results)	LR 11 (Design)
In-Lab	Task 1	-	-	/10
	Task 2	/20	/10	-
	Task 3	/20	/20	-
Total Points		**/40**	**/30**	**/10**
CLO Mapped		CLO 1		

Affective Domain Rubric		Points	CLO Mapped
AR7	**Report Submission & Git Upload**	/10 & /10	CLO 1

CLO	Total Points	Points Obtained
1	100	
Total	**100**	

For description of different levels of the mapped rubrics, please refer to the Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics provided here.